



Centro Universitário de Excelência Sistemas de Informação

CEARÁ

Autor: <Igor Flôres Veloso>
<Samir Santos Cabral>
<Jean Morais Dutra Júnior>
<Felipe Miranda dos Santos>
<Marco Douglas Pereira Ramos>

Agenda

O Objetivo é desenvolver um sistema modular em Python capaz de organizar e processar dados de um aplicativo de delivery, utilizando estruturas eficientes — como a árvore AVL — para tornar as consultas e o gerenciamento de pedidos mais rápidos e estruturados.

1. Introdução:

Este relatório apresenta o desenvolvimento de um sistema modular em Python para o Tia Lu Food App, criado com o objetivo de organizar e processar dados relacionados a restaurantes e pedidos. Com o uso de estruturas eficientes, como a árvore AVL, o sistema garante maior rapidez nas consultas e melhor desempenho no gerenciamento das informações do aplicativo de delivery.

2. Objetivo:

O objetivo do trabalho é criar um sistema capaz de organizar todas as informações do aplicativo de delivery — como menus, pedidos e dados dos restaurantes — de forma simples e eficiente. Para isso, o sistema foi dividido em módulos separados e utilizou estruturas como a árvore AVL, permitindo que as buscas e o processamento dos dados.

3. Metodologia:

O sistema foi desenvolvido de forma modular, dividindo cada função em arquivos específicos, como menu, pedidos, ordenação, armazenamento de dados e árvore AVL. Cada módulo executa uma tarefa própria, permitindo organização, fácil manutenção e integração eficiente entre as partes do sistema.

4. Resultados:

O sistema desenvolvido mostrou-se funcional e bem organizado, permitindo gerenciar menus e pedidos de forma eficiente. A estrutura modular facilitou a integração entre os componentes, e o uso da árvore AVL garantiu consultas mais rápidas, melhorando o desempenho geral do aplicativo de delivery.

5. Conclusão:

O desenvolvimento do sistema demonstrou a eficácia da abordagem modular e do uso de estruturas de dados avançadas, como a árvore AVL, para organizar e processar informações em um aplicativo de delivery. O projeto atendeu aos objetivos propostos, oferecendo uma solução funcional, escalável e capaz de melhorar o desempenho nas consultas e no gerenciamento de dados, servindo como base sólida para futuras expansões.

Fundamentação Teórica:



- **Árvores AVL**
- As árvores AVL são estruturas de dados binárias auto-balanceadas que garantem operações rápidas de busca, inserção e remoção. Elas mantêm o equilíbrio automático após cada modificação, permitindo que o acesso às informações aconteça em tempo $O(\log n)$. No projeto, essa estrutura é essencial para agilizar consultas dentro do aplicativo de delivery.
- **Estruturas de Ordenação**
- O módulo `ordenacao.py` implementa algoritmos para organizar listas, como restaurantes, pedidos e itens do menu. Esses métodos garantem que as informações sejam exibidas de forma organizada e eficiente, contribuindo para a experiência do usuário no aplicativo.

Fundamentação Teórica:



- **Armazenamento de Dados (JSON)**
- O sistema utiliza um arquivo dados.json como base de armazenamento. Esse formato é leve, estruturado e amplamente usado para organizar informações de forma clara. A leitura e manipulação desse arquivo são feitas pelo módulo storage.py, que conecta os dados aos demais componentes da aplicação.
- **Programação Modular**
- A aplicação foi construída seguindo o princípio de modularização, dividindo o sistema em arquivos independentes — como menu, pedidos, ordenação, AVL e armazenamento. Essa abordagem melhora a organização do código, facilita a manutenção e permite que novas funcionalidades sejam adicionadas sem afetar o restante da estrutura.

METODOLOGIA:



- **Estruturação Modular do Sistema**
- O desenvolvimento foi realizado dividindo o projeto em módulos independentes, cada um responsável por uma parte específica da aplicação. Essa estrutura facilita manutenção, testes e expansão futura.
- **Cada arquivo Python recebeu uma função clara:**
- main.py: coordena toda a aplicação.
- menu.py: gerencia a exibição e navegação do menu.
- pedido.py: cria e manipula pedidos.
- ordenacao.py: organiza dados em listas.
- storage.py: integra o sistema ao arquivo de dados.
- avl.py: implementa a árvore AVL usada nas buscas.

Resultados:



- O sistema desenvolvido demonstrou bom desempenho, apresentando organização, rapidez nas consultas e integração eficiente entre os módulos. A utilização da árvore AVL otimizou o acesso aos dados, enquanto a estrutura modular garantiu funcionamento claro e fácil manutenção.
- **Principais Resultados:**
- Funcionamento eficiente e bem estruturado
- Consultas rápidas com o uso da árvore AVL
- Integração completa entre os módulos do sistema
- Dados organizados e armazenados de forma estruturada (JSON)
- Facilidade na manipulação de menus e pedidos
- Arquitetura modular que permite expansão futura

Considerações finais



- O sistema desenvolvido demonstrou a eficiência da abordagem modular e do uso de estruturas de dados como a árvore AVL para otimizar consultas e organizar informações. A divisão das funções em módulos independentes facilitou a manutenção e garantiu um fluxo claro dentro da aplicação. O armazenamento em JSON contribuiu para uma leitura estruturada e integração simples entre os componentes. Os resultados mostraram um sistema funcional, organizado e capaz de atender às demandas de um aplicativo de delivery. O projeto cumpre seus objetivos e oferece uma base sólida para futuras melhorias e expansões.

Referências



- Referências:

Python Software Foundation. (2025). Python Documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: 16 set. 2025. CURSO EM VÍDEO. cursoemvideo/corsoemvideo-python [repositório no GitHub]. GitHub. Disponível em: <https://github.com/corsoemvideo/corsoemvideo-python>. Acesso em: 15 set. 2025. CURSO EM VÍDEO. Curso Python 3 – Mundo 1 [curso online]. Disponível em: <https://www.cursoemvideo.com/curso/python-3-mundo-1/>. Acesso em: 15 set. 2025.